



UNIVERSIDAD INTERNACIONAL DEL ECUADOR

FACULTAD DE CIENCIAS TÉCNICAS

CARRERA DE INGENIERÍA EN CIBERSEGURIDAD

AUTÓNOMO 1

PROYECTO INTEGRADOR

TEMA DE EVALUACIÓN:

Sistema de Gestión de Libros Electrónicos bajo el Paradigma de Programación Funcional en Golang: Etapa 1. Planeación del Software

Profesor: Héctor Guillermo Avalos Silva

Asignatura: Introducción a las Redes de Datos

Autores: Valenzuela Saavedra Andrés Fernando

Fecha: 14 de agosto del 2025



Índice

1	Introducción y Objetivos del Sistema	2
1.1	Objetivo General.....	2
1.2	Objetivos Específicos	2
2	Alcance del Proyecto	2
2.1	Límites Funcionales (Alcance Dentro del Proyecto).....	2
2.2	Límites No Funcionales y Exclusiones (Fuera del Alcance)	3
3	Estructura de Funcionalidades por Módulo	3
3.1	Descripción Operativa de los Módulos	4
4	Arquitectura de Software y Programación Funcional en Go	4
4.1	Conceptos Clave Aplicados.....	4
4.2	Implementación de Funciones de Orden Superior Genéricas.....	4
4.3	Composición Criptográfica Funcional	5
5	Paquetes y Dependencias (Ecosistema de Software)	6
5.1	Paquetes de la Biblioteca Estándar de Go (Sin dependencias externas).....	6
5.2	Paquetes de Terceros Documentados	6
6	Configuración del Repositorio de GitHub	6
6.1	Estructura del Proyecto en el Repositorio	7
6.2	Pipeline de Integración Continua (GitHub Actions).....	7
7	Características de Entrega y Planificación del Video	8
7.1	Estructura Planificada del Video Explicativo	8



1 Introducción y Objetivos del Sistema

El auge de la transformación digital ha consolidado a los documentos y libros electrónicos como los principales portadores de conocimiento en entornos académicos y corporativos. Sin embargo, la distribución indiscriminada y la falta de mecanismos avanzados de control e integridad exponen a las organizaciones a graves brechas de seguridad informática, como la alteración de propiedad intelectual y fugas de información sensible.

Para solventar esta problemática, se proyecta el diseño e implementación del **Sistema de Gestión de Libros Electrónicos**. Desde una perspectiva de ciberseguridad moderna, el sistema se concibe bajo el paradigma de **Programación Funcional (FP)** utilizando el lenguaje de programación **Golang (Go)**. La elección de este enfoque reside en que la programación funcional promueve la inmutabilidad de los datos y la ausencia de efectos secundarios, lo cual erradica por completo condiciones de carrera (*race conditions*) en entornos concurrentes y reduce la superficie de ataque derivada de la manipulación inadecuada del estado en memoria.

1.1 Objetivo General

Diseñar y planificar un sistema seguro de gestión de libros electrónicos empleando el paradigma de programación funcional en Golang para garantizar la inmutabilidad, la trazabilidad del contenido y un procesamiento concurrente seguro contra vectores de ataque de concurrencia y corrupción de memoria.

1.2 Objetivos Específicos

- **Seguridad de Datos e Integridad:** Establecer funciones puras deterministas para la validación de sumas de comprobación criptográficas (SHA-256) que prevengan alteraciones maliciosas en los libros digitales.
- **Arquitectura Funcional Robustecida:** Estructurar el núcleo lógico del software mediante pipelines de funciones y estructuras de datos inmutables, minimizando estados compartidos.
- **Gestión de Control y Acceso:** Definir un módulo de autenticación criptográfica que utilice firmas digitales basadas en tokens inmutables (JWT), aislando los efectos secundarios del entorno de red.
- **Auditoría de Operaciones:** Diseñar un registro (*ledger*) transaccional inmutable que registre de forma transparente todas las lecturas, subidas y descargas efectuadas en la plataforma.

2 Alcance del Proyecto

El alcance del proyecto abarca el análisis, diseño estructural e implementación conceptual del Backend de la plataforma en su Etapa 1. A continuación, se detallan los límites funcionales y técnicos.

2.1 Límites Funcionales (Alcance Dentro del Proyecto)

1. **Procesamiento Funcional de Archivos:** Lectura analítica de metadatos de archivos de libros electrónicos (formato PDF/EPUB) a través de pipelines puros que generen representaciones estructuradas en memoria sin modificar los archivos físicos.



2. **Búsqueda y Filtrado Cero-Mutación:** Consultas multicriterio sobre el catálogo de libros mediante implementaciones genéricas de Map, Filter y Reduce, asegurando búsquedas rápidas libres de estados bloqueantes.
3. **Cifrado Criptográfico Homogéneo:** Encriptación simétrica AES-GCM del contenido de los libros antes de ser persistidos, de manera que solo los usuarios autorizados posean la función de transformación para su descifrado.
4. **Registro Histórico Determinista:** Registro inalterable de auditoría estructurado en un histórico persistente de transacciones.

2.2 Límites No Funcionales y Exclusiones (Fuera del Alcance)

- **Interfaz Gráfica de Usuario (GUI) Compleja:** El software se operará inicialmente mediante interfaces de programación (APIs REST funcionales) y herramientas de consola CLI.
- **Distribución de Archivos peer-to-peer (P2P):** El alcance está restringido a una arquitectura cliente-servidor concurrente protegida.

3 Estructura de Funcionalidades por Módulo

El sistema de gestión se divide en cuatro módulos principales. Cada módulo está modelado matemáticamente como un conjunto de transformaciones puras de entrada y salida, asegurando la predictibilidad.

Cuadro 1: Funcionalidades y Flujos de Datos por Módulo

Módulo	Entrada	Procesamiento (Función)	Salida (Retorno)
1. Catálogo Inmutable	Slice de Libros, Criterios de filtrado, Funciones de predicado.	Filtrado declarativo (Filter) y transformación de estructuras (Map).	Lista inmutable filtrada de libros estructurados.
2. Integridad y Criptografía	Archivo binario de libro ([]byte), clave criptográfica.	Cifrado simétrico determinista y cálculo de SHA-256 mediante composición.	Archivo cifrado con metadatos de integridad incorporados.
3. Acceso y Autorización	Token JWT entrante, firma asimétrica de verificación.	Decodificación y verificación de claims mediante clausuras (closures).	Estado de autorización (Contexto inmutable de usuario).
4. Auditoría y Registro	Registro de transacción (AuditLog), ledger actual.	Composición secuencial para adjuntar registro y actualización de hash raíz.	Nuevo estado del historial inalterable.



3.1 Descripción Operativa de los Módulos

- **Módulo 1: Catálogo e Inventario Inmutable.** Evita por completo la mutación del inventario en memoria global. Al añadir un libro, se computa una nueva colección que contiene la combinación de los elementos anteriores más el nuevo, asegurando la consistencia referencial.
- **Módulo 2: Integridad de Archivos.** Toma el flujo binario del archivo, calcula su firma digital para evitar el repudio de datos, y lo empaqueta con cifrado AES-256 en modo GCM, garantizando autenticidad y confidencialidad.
- **Módulo 3: Control de Acceso Cero-Estado.** El servidor no retiene variables de sesión en memoria activa. Las credenciales de acceso se validan procesando tokens firmados asimétricamente. Las autorizaciones se propagan mediante firmas funcionales.
- **Módulo 4: Registro Ledger de Auditoría.** Implementa el concepto de estructura de datos persistente (*persistent data structure*). Cada acción efectúa un paso incremental en una cadena inalterable, donde cualquier alteración en un registro intermedio invalida el hash del nodo subsecuente.

4 Arquitectura de Software y Programación Funcional en Go

Go no es un lenguaje puramente funcional en su concepción estándar; sin embargo, cuenta con un soporte de primer nivel para funciones de primera clase (*first-class functions*), clausuras (*closures*), funciones anónimas y, desde la versión 1.18, tipos genéricos. Esto permite implementar un motor robusto y óptimo de programación funcional.

4.1 Conceptos Clave Aplicados

1. **Inmutabilidad Estructural:** Los objetos del dominio (Book, User) no se modifican a través de punteros mutables. En su lugar, las funciones toman estructuras de valor y retornan nuevas copias modificadas.
2. **Funciones de Orden Superior (Higher-Order Functions):** Funciones que toman otras funciones como parámetros de filtrado o transformación.
3. **Pipelines mediante Canales:** Utilización de canales Go concurrentes para canalizar datos binarios mediante transformadores criptográficos puros, previniendo condiciones de carrera por diseño.

4.2 Implementación de Funciones de Orden Superior Genéricas

A continuación, se detalla la implementación en Go del módulo funcional genérico que procesará las colecciones de libros sin generar efectos colaterales en la memoria física:

```
1 package fp
2
3 // Book representa la estructura inmutable del Libro Electronico
4 type Book struct {
5     ID        string
6     Title     string
7     Author    string
8     Checksum  string
```



```
9     IsSecured bool
10 }
11
12 // Filter filtra un slice en funcion de un predicado puro.
13 func Filter[T any](slice []T, predicate func(T) bool) []T {
14     var result []T
15     for _, v := range slice {
16         if predicate(v) {
17             result = append(result, v)
18         }
19     }
20     return result
21 }
22
23 // Map transforma un slice aplicando una funcion transformadora pura.
24 func Map[T, U any](slice []T, transform func(T) U) []U {
25     result := make([]U, len(slice))
26     for i, v := range slice {
27         result[i] = transform(v)
28     }
29     return result
30 }
```

Listing 1: Funciones de Orden Superior Genéricas para el Catálogo Inmutable

4.3 Composición Criptográfica Funcional

En ciberseguridad, los flujos de procesamiento deben ser deterministas. El siguiente fragmento ejemplifica cómo concatenar operaciones puras de seguridad (Cálculo de Hash y Cifrado Simulado) mediante composición funcional en Go:

```
1 package security
2
3 import (
4     "crypto/sha256"
5     "encoding/hex"
6 )
7
8 // Pipeline define la firma de transformacion de un libro
9 type Pipeline func(data []byte) []byte
10
11 // ComputeSHA256 es una funcion pura para calcular el hash del archivo
12 func ComputeSHA256(data []byte) string {
13     hash := sha256.Sum256(data)
14     return hex.EncodeToString(hash[:])
15 }
16
17 // Compose combina multiples operaciones de transformacion criptografica
18 func Compose(transforms ...Pipeline) Pipeline {
19     return func(data []byte) []byte {
20         result := data
21         for _, transform := range transforms {
22             result = transform(result)
23         }
24         return result
25     }
26 }
27
```



```
28 // EncriptarXOR es una transformacion pura determinista de ejemplo
29 func EncriptarXOR(key byte) Pipeline {
30     return func(data []byte) []byte {
31         encrypted := make([]byte, len(data))
32         for i, b := range data {
33             encrypted[i] = b ^ key
34         }
35         return encrypted
36     }
37 }
```

Listing 2: Pipeline Criptográfico Funcional para Procesamiento Seguro

5 Paquetes y Dependencias (Ecosistema de Software)

Para garantizar el cumplimiento de los requerimientos de la tarea, se listan los paquetes del ecosistema de software a utilizar, detallando su justificación de uso y licencias:

5.1 Paquetes de la Biblioteca Estándar de Go (Sin dependencias externas)

- `crypto/sha256` y `crypto/aes`: Requeridos para la implementación de las funciones puras de hashing e integridad, y cifrado simétrico robusto AES-GCM.
- `encoding/json`: Permite realizar la serialización de registros inmutables de auditoría en formato estandarizado.
- `net/http`: Utilizado para exponer el catálogo inmutable como un servicio de red seguro (REST API).

5.2 Paquetes de Terceros Documentados

- **JWT para Go** (github.com/golang-jwt/jwt/v5):
 - *Uso*: Generación e inspección determinista de claims de autenticación firmados criptográficamente.
 - *Justificación*: Permite una validación descentralizada y libre de estado (*stateless*), alineándose con los principios funcionales.
 - *Licencia*: MIT.
- **Argon2 para Contraseñas** (golang.org/x/crypto/argon2):
 - *Uso*: Derivación de claves criptográficas seguras a partir de credenciales de usuario.
 - *Justificación*: Estándar recomendado por la OWASP contra ataques de fuerza bruta y hardware optimizado (ASICs), estructurado como función pura en Go.
 - *Licencia*: BSD-style (Go License).

6 Configuración del Repositorio de GitHub

Un flujo de trabajo estandarizado es vital para certificar la calidad y robustez del software. Se ha estructurado el repositorio siguiendo las mejores prácticas de arquitectura limpia y acoplamiento funcional en Go.



6.1 Estructura del Proyecto en el Repositorio

La distribución de directorios se apega a la convención estándar del ecosistema Go:

```
1 uide -ebook - manager/  
2 .github/  
3     workflows/  
4         ci-cd.yml          # Pipeline Automatizado (Linter, Testing, Gosec  
5     )  
6 cmd/  
7     ebook-manager/  
8         main.go           # Punto de entrada de la aplicacion  
9 pkg/  
10    domain/  
11        book.go           # Estructuras de datos puras e inmutables  
12    fp/  
13        functional.go      # Motor de Map, Filter y Reduce  
14    security/  
15        crypto.go         # Funciones puras de cifrado y hashing  
16        auth.go           # Verificacion de tokens inmutables  
17    audit/  
18        ledger.go         # Historial de auditoria determinista  
19 go.mod                   # Declaracion del modulo Go  
    README.md             # Documentacion de la planificacion y  
    despliegue
```

Listing 3: Estructura Inmuta-Ebook en GitHub

6.2 Pipeline de Integración Continua (GitHub Actions)

Para validar la compilación automática y ejecutar pruebas estáticas de seguridad, se ha planificado la configuración del archivo ci-cd.yml:

```
1 # Pipeline de Integracion Continua para Sistema de Libros  
2 name: Go Functional CI  
3  
4 on:  
5   push:  
6     branches: [ "main", "develop" ]  
7   pull_request:  
8     branches: [ "main" ]  
9  
10 jobs:  
11   build -and -test:  
12     runs-on: ubuntu -latest  
13     steps:  
14       - name: Checkout Code  
15         uses: actions/checkout@v4  
16  
17       - name: Setup Go Environment  
18         uses: actions/setup-go@v5  
19         with:  
20           go-version: '1.21'  
21  
22       - name: Restore Go Cache  
23         uses: actions/cache@v3  
24         with:  
25           path: ~/go/pkg/mod  
26           key: ${{ runner.os }}-go-${{ hashFiles('**/go.sum') }}
```




```
27
28 - name: Install Dependencies
29   run: go mod download
30
31 - name: Run Security Static Analysis (gosec)
32   run: |
33     go install github.com/securego/gosec/v2/cmd/gosec@latest
34     gosec ./...
35
36 - name: Run Unit Tests for Functional Code
37   run: go test -v -race ./...
```

Listing 4: Archivo de Integración Continua .github/workflows/ci-cd.yml

7 Características de Entrega y Planificación del Video

La entrega final del producto comprende tres entregables clave: el documento de especificación formal (este archivo), el repositorio de GitHub configurado y un video descriptivo.

7.1 Estructura Planificada del Video Explicativo

El video de sustentación técnica tendrá una duración sugerida de entre 5 y 8 minutos, estructurado con la siguiente agenda académica:

1. **Introducción (1 Minuto):** Presentación del autor (Andrés Fernando Valenzuela Saavedra), contextualización de la carrera de Ciberseguridad de la UIDE y explicación del problema a resolver.
2. **Justificación Tecnológica (1.5 Minutos):** Sustento de por qué se eligió Golang acoplado con programación funcional y cómo la inmutabilidad de datos y las funciones puras mitigan riesgos y vulnerabilidades típicas en sistemas imperativos ordinarios.
3. **Arquitectura y Estructura Funcional (2 Minutos):** Explicación de los diagramas conceptuales de los módulos de Catálogo, Seguridad, Control de Acceso y Auditoría Transaccional.
4. **Demostración de Código y Repositorio (2 Minutos):** Recorrido por la estructura del repositorio de GitHub, demostrando los flujos automatizados de GitHub Actions (CI/CD) y enseñando el código funcional implementado para los métodos genéricos Filter y Map.
5. **Conclusiones (0.5 Minutos):** Cierre de la presentación vinculando la teoría de redes de datos y criptografía de contenido con los entregables de la Etapa 1.

8 Referencias

- Bodner, J. (2021). Learning Go: An idiomatic approach to real-world Go programming. O'Reilly Media.
- Donovan, A. A., y Kernighan, B. W. (2015). The Go programming language. Addison-Wesley Professional.
- Dworkin, M. (2007). Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC (NIST Special Publication 800-38D). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-38D>
- Jones, M., Bradley, J., y Sakimura, N. (2015). JSON Web Token (JWT) (RFC 7519). Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc7519>



Lipovača, M. (2011). Learn you a Haskell for great good!: A beginner's guide. No Starch Press.
OWASP Foundation. (2021). OWASP Top 10:2021 - Los riesgos de seguridad más críticos en aplicaciones web. <https://owasp.org/www-project-top-ten/>